TASK # 1

Use more than 3 columns/features as your independent variables from any data set you want and find out the best parameters αi for the model given by equation (1) that fits the data. Use SGD to find parameters.

The data which is used in the following tasks has been taken from the kaggle. The dataset is of USA Housing Dataset which includes 6 columns including target variable "Price". In this task we will predict the house prices in USA using Stochastic gradient descent.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
```

```
import numpy as np
import pandas as pd
import plotly.express as px
!pwd
os.chdir('/content/drive/My Drive')
```

```
/content/drive/My Drive
```

```
df = pd.read_csv('USA_Housing(lab).csv')
```

```
len(df.columns)
```

```
6
```

```
df.shape
```

```
(1999, 6)
```

```
df
```

	Area_Income	Area_HouseAge	Area_NumberofRooms	Area_NumberofBedrooms	Area_Population	Price
0	79545.45857	5.682861	7.009188	4.09	23086.80050	1.059034e+06
1	79248.64245	6.002900	6.730821	3.09	40173.07217	1.505891e+06
2	61287.06718	5.865890	8.512727	5.13	36882.15940	1.058988e+06
3	63345.24005	7.188236	5.586729	3.26	34310.24283	1.260617e+06
4	59982.19723	5.040555	7.839388	4.23	26354.10947	6.309435e+05
...	...	...	...	...	...	...
1994	79710.19507	5.093114	8.241880	6.24	39830.05126	1.448668e+06
1995	52799.78886	6.746739	7.315886	6.50	20438.41484	7.284020e+05
1996	64634.63095	7.065144	4.583646	2.35	52131.81151	1.381483e+06
1997	69467.90130	4.887027	7.345657	4.10	55700.44166	1.424994e+06
1998	79504.65959	6.505109	7.554076	4.10	42677.28492	1.741106e+06

1999 rows × 6 columns

```
x1 = np.array(df.Area_Income, dtype=float)
x2 = np.array(df.Area_HouseAge, dtype=float)
x3 = np.array(df.Area_Population, dtype=float)
x4 = np.array(df.Area_NumberofRooms, dtype=float)
x5 = np.array(df.Area_NumberofBedrooms, dtype=float)
b = np.array(df.Price, dtype=float)

X0 = np.ones(x1.shape, dtype = float )
```

Shifting all the Data Columns to have a Zero mean value and unit variance

```
mid = int(X0.shape[0]/2)
X0[0:mid-1]=-1
X1 = (x1-x1.mean())/x1.std()
X2 = (x2-x2.mean())/x2.std()
X3 = (x3-x3.mean())/x3.std()
X4 = (x4-x4.mean())/x4.std()
X5 = (x5-x5.mean())/x5.std()
A_stan = np.array([X0,X1,X2,X3,X4,X5]).T
B = b.reshape(b.shape[0],1)
B = (B-B.mean())/B.std()
```

```
Xopt = np.linalg.inv(A_stan.T.dot(A_stan)).dot(A_stan.T).dot(B)   # Normal equations
```

```
u = 0.001 # The Learning Rate or the Step-Size
N = 1000  # Number of times the learning process repeated to be averaged afterwards
n = 1500 # Number of Data points used to reach the minimum of the cost function
Total_cost = np.empty(n)
for k in range(N):
  Cost = np.empty(n)
  Alpha = np.zeros((6,1),dtype=float)
  for i in range(n):
    j = np.random.randint(0, n - 1) # A Random index to pick a Random Data point
    x = A_stan[j].reshape(6,1)      # A row of matrix 'A' is taken which represents a single data point
    e = B[j] - np.dot(Alpha.T,x)    # Error = y - aT.x
    Alpha = Alpha + u*e*x
    Cost[i] = 1/2*e**2            # Cost Accumulation
                                   #Cost = Cost/2
  Total_cost += Cost                # Cost Accumulation for an Epoch
Total_cost = Total_cost/N
```

```
<ipython-input-496-5e48794de6c5>:13: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will er
  Cost[i] = 1/2*e**2            # Cost Accumulation
```

```
Alpha
```

```
array([[-0.00770653],
       [ 0.504437  ],
       [ 0.34514214],
       [ 0.31785763],
       [ 0.25092159],
       [ 0.06119274]])
```

```
Xopt
```

```
array([[-0.00351352],
       [ 0.65607853],
       [ 0.46314157],
       [ 0.40947809],
       [ 0.33323494],
       [-0.00078927]])
```

TASK # 2

Write the model equation after finding the parameters $\boldsymbol{\alpha i}$.

EQUATIONS:

The approximate Least-squares solution estimating the parameters.Hence, the model

$$y = -0.007x_0 + 0.504x_1 + 0.345x_2 + 0.317x_3 + 0.250x_4 + 0.061x_5$$

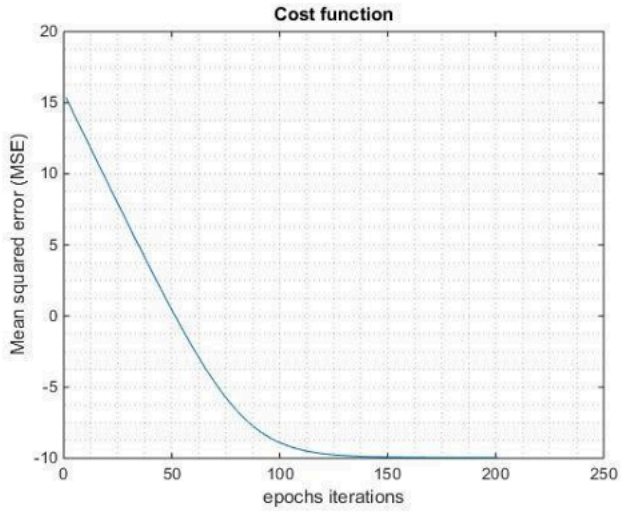
$$= \begin{bmatrix} -0.001 & 0.504 & 0.345 & 0.317 & 0.250 & 0.061 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \alpha^T \mathbf{x}$$

The Least-squares solution estimating the parameters α_0, α_1, α_2, α_3 ,α_4 and α_5 achieved using the Normal Equations, Hence, the model $y = -0.0035x_0 + 0.6560x_1 + 0.463x_2 + 0.409x_3 + 0.333x_4 - 0.0007x_5$

$$= \begin{bmatrix} -0.0035 & 0.6560 & 0.463 & 0.409 & 0.333 & -0.0007 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \alpha^T \mathbf{x}$$

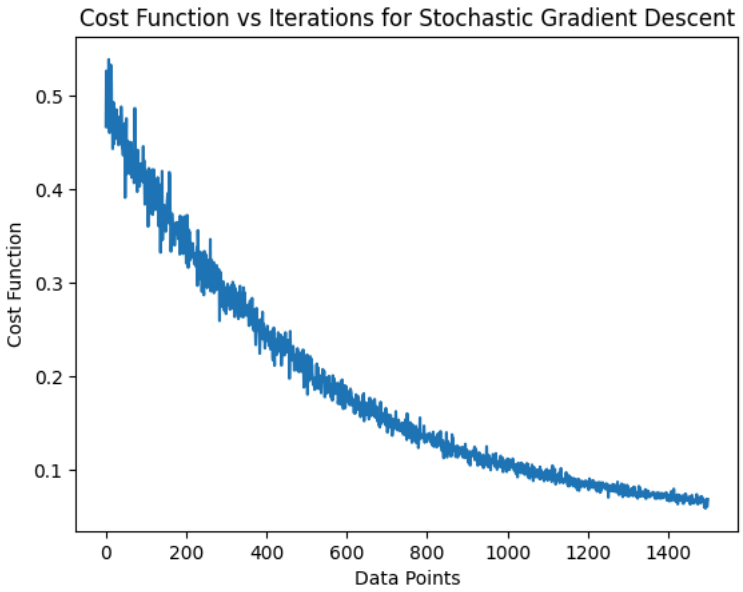
TASK # 3

Plot the Cost function vs the Time/Iterations to show how the error decreases similar to the following graph:



Plotting the graph for Cost/ Iterations:

```
import numpy as np
import pandas as pd
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
plt.plot(Total_cost)
plt.xlabel("Data Points")
plt.ylabel("Cost Function")
plt.title("Cost Function vs Iterations for Stochastic Gradient Descent")
plt.show()
```



This graph has a similar response to the required one. Thus, its can be concluded that the algorithm reaches to a minimum point i.e., we have implemented the algorithm or have set the parameters in an optimal manner.